



Guia do Aluno — Fechando a Semana com o MVC

UC: Programação Back-End | **Bloco 2 — Aula de 28/08/2026**

Projeto: API de Gestão da Livraria (SA1)

O que hoje é

Hoje é dia de terminar o que ficou faltando da última aula e conferir se o diagrama que vocês desenharam em na aula de diagramas UML ainda bate com o código que existe de verdade.

1. O fluxo, de novo, com um pedido diferente

Até agora o exemplo sempre foi "buscar os livros" (`GET /livros`). Hoje o exemplo muda para **cadastrar um livro novo** (`POST /livros`) — o fluxo pelas 4 camadas é o mesmo, só a ação é diferente.

"Cadastrar um livro novo"

Cliente envia os dados do livro → ROTA (`POST /livros`)

- CONTROLLER (o que fazer com um cadastro?)
 - SERVICE (valida os dados, decide se pode c
 - MODEL (onde o livro seria guardado)
 - ← confirma que deu certo
 - ← controller devolve a resposta

Cliente recebe a confirmação

Repare: a estrutura é idêntica à de `GET /livros` . Isso é o ponto principal de hoje — o fluxo rota → controller → service → model não muda dependendo da ação. Muda só o que cada camada faz por dentro.

💡 Se travar em algum ponto, a frase que já usamos na última aula continua valendo: **"quem decide não é quem executa."**

2. Finalizar o esqueleto

Se o seu grupo ainda não fez o desafio extra de 19/08, hoje é a hora de completar. Dentro do repositório do grupo, criem os 3 arquivos que faltam para `Categoria`, seguindo exatamente o padrão que já existe para `Livro`:

`src/routes/categoriaRoutes.js`

```
// ROTA: recebe a requisicao HTTP.
// Aqui vao ficar os caminhos (endpoints) relacionados a Categoria.
// Ex: GET /categorias, POST /categorias
// Implementacao chega no Bloco 3, quando o banco de dados entrar.

module.exports = {};
```

`src/controllers/categoriaController.js`

```
// CONTROLLER: decide o que fazer com cada pedido de Categoria.
// Implementacao chega no Bloco 3.

module.exports = {};
```

`src/services/categoriaService.js`

```
// SERVICE: executa a logica de verdade sobre categorias.
// Implementacao chega no Bloco 3.

module.exports = {};
```

Confiram que o servidor ainda sobe normalmente:

```
npm run dev
```

3. Auditoria: o diagrama ainda bate com o código?

Esta é a atividade principal de hoje. Vocês fizeram um diagrama UML em 14/08 com 10 classes. Hoje, comparem esse diagrama com o que existe de verdade em `src/models/` do repositório do grupo.

Vão encontrar **duas situações diferentes de "falta"**, e é importante saber diferenciar:

Situação 1 — falta porque ainda não foi consolidada. É o caso de `LivroFisico`, `LivroDigital` e `Carrinho`. Essas classes já existem, escritas, nos repositórios pessoais de algum integrante — só não foram copiadas para o repositório do grupo ainda. É uma pendência simples de resolver.

Situação 2 — falta porque a entidade ainda não existe em código nenhum. É o caso de `Pedido` e `ItemPedido`. Essas classes foram desenhadas em 14/08 de propósito, antes de existir qualquer código — porque UML serve exatamente para modelar antes de implementar. Elas vão virar código só no Bloco 3, quando o banco de dados entrar. Não é uma pendência para resolver hoje.

Sigam a atividade `03-atividade-auditoria-mvc-uml.md` para montar essa comparação em tabela.

Checklist

- ☐ `categoriaRoutes.js`, `categoriaController.js`, `categoriaService.js` criados, se ainda faltavam
- ☐ Servidor ainda roda normalmente com `npm run dev`
- ☐ Consigo narrar o fluxo de `POST /livros` pelas 4 camadas, sem ajuda
- ☐ Tabela de auditoria preenchida, com as 10 classes do diagrama
- ☐ Sei explicar a diferença entre as duas situações de "falta" descritas na seção 3
- ☐ `git push` feito

Cola rápida

ROTA	→ recebe o pedido
CONTROLLER	→ decide o que fazer
SERVICE	→ executa a logica
MODEL	→ guarda os dados

```
npm run dev          # confirma que o servidor ainda sobe
git add .
git commit -m "feat: finaliza esqueleto MVC e audita coerencia com o diagrama UML"
git push
```